

DMD su Raspberry Pi — Manuale completo

Versione del software: 1.8 · Repository: <https://github.com/kWGillo/zedmd-pi>

Pannelli LED P2.5 128×64 con chip **FM6373** (S-PWM), due moduli in cascata → **256×64 px**, pilotati da un Raspberry Pi che si presenta a Batocera come un **ZeDMD-WiFi**.

Questo documento copre l'intera messa in opera, **dal primo avvio del Raspberry fino al sistema funzionante**, aggiornamenti compresi. Sostituisce i due manuali separati precedenti.

Punto di partenza: microSD già scritta con Raspberry Pi OS Lite (64 bit; per la Pi Zero W la versione **32 bit**), con Wi-Fi, utente e SSH già impostati da Raspberry Pi Imager. In tutto il documento l'utente è `gillo` e l'hostname è `dmdpi`.

Indice

1. Prima di cominciare
2. Primo avvio e accesso SSH
3. Aggiornamento del sistema
4. Preparazione per il pannello
5. Libreria della matrice
6. Cablaggio
7. Verifica del pannello
8. Installazione del DMD Controller da GitHub
9. Configurazione iniziale
10. Avvio e interfaccia web
11. Collegamento a Batocera
12. Musica: il brano in ascolto sul pannello
13. Aggiornamenti
14. Installazione resistente all'usura
15. Righe bianche, rallentamenti e blocchi
16. Risoluzione problemi
17. Comandi utili e struttura dei file
18. Appendice A — diagnostica del pannello
19. Appendice B — pubblicare una nuova versione su GitHub

1. Prima di cominciare

1.1 Perché questa procedura esiste

I pannelli montano il driver **FM6373**, un chip S-PWM con framebuffer integrato. Non è supportato né da ESP32-HUB75-MatrixPanel-DMA — e quindi **non da ZeDMD** — né dalla libreria ufficiale `hzeller/rpi-rgb-led-matrix`.

È supportato **solo** dal fork sperimentale `kingdo9/rpi-rgb-led-matrix_pwm_experiment`, che funziona **esclusivamente su Raspberry Pi**: scrive direttamente sui registri Broadcom/RP1, quindi Orange Pi, Radxa, Banana Pi e simili sono esclusi in partenza.

In più i pannelli espongono solo le linee di indirizzo **A, B, C** — D ed E sono serigrafate NC — e richiedono un profilo di registro specifico, individuato sperimentalmente.

Il DMD Controller mette quel fork dietro il protocollo di rete di ZeDMD, così Batocera crede di parlare con un dispositivo reale.

1.2 Scelta del modello

Modello	Core	slowdown	isolcpus	Giudizio
Pi Zero W (v1.1)	1 × ARM11	1 (provare 2)	non applicabile	Funziona, ma senza core dedicato. Adatto a contenuti statici e slideshow; margine

				ridotto con streaming DMD, web e video insieme.
Pi Zero 2 W	4 × A53	3	isolcpus=3	Buon compromesso ingombro/prestazioni. Solo Wi-Fi 2.4 GHz.
Pi 3 (3A+/3B/3B+)	4 × A53	3	isolcpus=3	Come la Zero 2 W ma con più RAM e più porte. Facile da reperire.
Pi 4	4 × A72	5	isolcpus=3	Il più capace, margine abbondante per tutto insieme. Scalda: prevedere dissipatore. Richiede un USB-C 5V/3A serio.

Il **cablaggio è identico su tutti i modelli**: l’header a 40 pin ha la stessa disposizione. Sulle Zero l’header potrebbe non essere saldato di fabbrica.

Nei comandi che seguono compare la variabile `$SLOW`. Impostala **all’inizio di ogni sessione SSH** con il valore della tabella:

```
export SLOW=3
```

Per la Pi Zero W `export SLOW=1`, per la Pi 4 `export SLOW=5`.

1.3 La scheda SD

Prima di procedere, una raccomandazione che nasce da un guasto reale: usa una scheda della linea **high endurance** (SanDisk High Endurance o Max Endurance, Samsung PRO Endurance). Le sigle V10, V30, A1 e “100 MB/s” misurano la velocità, non la durata, e un Raspberry acceso sempre logora una scheda comune nel giro di mesi. Bastano 32 GB. La sezione 14 spiega come ridurre l’usura.

2. Primo avvio e accesso SSH

Inserisci la microSD e alimenta il Pi:

- Zero W / Zero 2 W: porta micro-USB marcata **PWR**
- Pi 3: micro-USB, alimentatore 5V/2.5A
- Pi 4: USB-C, alimentatore 5V/3A

Attendi due o tre minuti — il primo avvio espande il filesystem e riavvia da solo — poi dal Mac:

```
ssh gillo@dmdpi.local
```

Se il nome non si risolve, cerca l’indirizzo IP nella pagina dei dispositivi collegati del router e usa quello.

Non spegnere mai il Pi togliendo corrente. Usa `sudo poweroff`, attendi che il LED verde smetta di lampeggiare, poi stacca. Lo spegnimento brusco è una delle cause più comuni di corruzione della scheda SD.

3. Aggiornamento del sistema

Da fare subito, e da ripetere ogni tanto nel tempo.

```
sudo apt update && sudo apt full-upgrade -y
```

```
sudo apt install -y git build-essential
```

Sulla **Pi 4** conviene aggiornare anche il firmware dell’EEPROM, che migliora stabilità e gestione termica:

```
sudo rpi-eeprom-update -a && sudo reboot
```

Sugli altri modelli il comando non serve.

Quando il DMD è già in funzione, prima di un aggiornamento di sistema ferma il servizio: `sudo systemctl stop dmd`. Un `apt full-upgrade` occupa disco e CPU, e il pannello mostrerebbe artefatti per tutta la durata. Al termine, `sudo systemctl start dmd`.

4. Preparazione per il pannello

4.1 Disattivare l'audio integrato

Obbligatorio su tutti i modelli: l'audio onboard usa lo stesso hardware PWM che serve a pilotare i pannelli.

```
sudo sed -i 's/^dtparam=audio=on/dtparam=audio=off/' /boot/firmware/config.txt
```

```
echo "blacklist snd_bcm2835" | sudo tee /etc/modprobe.d/blacklist-audio.conf
```

4.2 Riservare un core della CPU

Solo su Pi Zero 2 W, Pi 3 e Pi 4. Sulla Pi Zero W questo passo va **saltato**: ha un solo core e isolarlo bloccherebbe il sistema.

```
sudo sed -i 's/^isolcpus=3/' /boot/firmware/cmdline.txt
```

4.3 Verifica e riavvio

```
grep audio /boot/firmware/config.txt && cat /boot/firmware/cmdline.txt
```

`config.txt` deve contenere `dtparam=audio=off`. Sui modelli quad-core `cmdline.txt` deve terminare con `isolcpus=3`, **su una sola riga**: se il comando l'avesse spezzata, correggila con `sudo nano` prima di riavviare.

```
sudo reboot
```

5. Libreria della matrice

```
git clone https://github.com/kingdo9/rpi-rgb-led-matrix_pwm_experiment.git
```

```
cd rpi-rgb-led-matrix_pwm_experiment && make
```

```
cd examples-api-use && make && cd ~
```

Tempi indicativi: pochi minuti su Pi 4 e Pi 3, qualche minuto in più sulla Zero 2 W, sensibilmente di più sulla Zero W.

6. Cablaggio

Tutto rigorosamente a dispositivi spenti. Identico su tutti i modelli.

6.1 Segnali dati

Mappatura “regular” della libreria hzeller. Pin fisici contati sull’header a 40 poli: pin 1 all’angolo lato microSD, dispari sulla fila interna, pari su quella esterna.

Segnale pannello	GPIO (BCM)	Pin fisico Pi
R1	GPIO 11	23
G1	GPIO 23	13

G1	GPIO 27	13
B1	GPIO 7	26
R2	GPIO 8	24
G2	GPIO 9	21
B2	GPIO 10	19
A	GPIO 22	15
B	GPIO 23	16
C	GPIO 24	18
CLK	GPIO 17	11
LAT	GPIO 4	7
OE	GPIO 18	12
GND	—	6 e 14

D ed E non si collegano (NC sul pannello). I GPIO che la libreria vi assocerebbe, 25 e 15, restano liberi.

6.2 Cascata dei due pannelli

Pi → connettore **JIN** (ingresso) del primo pannello; **JOUT** (uscita) del primo → **JIN** del secondo. Ogni pannello ha la propria alimentazione 5V.

6.3 Alimentazione — tre regole non negoziabili

1. I pannelli si alimentano da un **alimentatore 5V esterno** tramite i propri cavi di potenza (circa 4 A per pannello), **mai dal Pi**.
2. Il Pi si alimenta dal proprio alimentatore, secondo il modello (§2).
3. **Le masse devono essere in comune:** il GND dell'alimentatore dei pannelli e il GND del Pi (pin 6/14 → GND del connettore HUB75) devono essere elettricamente uniti.

I segnali del Pi sono a 3,3 V mentre il pannello ne attende 5: nella pratica funziona direttamente. Un level-shifter serve solo se compaiono sfarfallii persistenti.

Alimentatori separati per Pi e pannelli. Condividere l'alimentatore sembra comodo, ma sotto carico i LED fanno scendere la tensione sul Pi, e sotto i 4,65 V il controller della scheda SD si resetta a metà scrittura. Si verifica in qualsiasi momento con `vcgencmd get_throttled`: deve rispondere `throttled=0x0`.

7. Verifica del pannello

7.1 Parametri funzionanti

Questi valori sono il risultato di una campagna diagnostica e vanno usati **tutti insieme**.

Parametro	Valore	Ruolo
<code>--led-rows</code>	64	righe del pannello
<code>--led-cols</code>	128	colonne del singolo pannello
<code>--led-chain</code>	2	pannelli in cascata (1 per il test singolo)
<code>--led-panel-type</code>	fm6373	driver S-PWM
<code>--led-spwm-row-addr-type</code>	1	indirizzamento righe a shift register
<code>--led-spwm-scan</code>	64	critico: con 32 l'immagine si duplica
<code>--led-spwm-register-config</code>	2	critico: profilo di registro del chip
<code>--led-no-drop-privs</code>	—	critico: senza, il catalogo profili è illeggibile
<code>--led-slowdown-gpio</code>	\$SLOW	dipende dal modello (§1.2)
<code>--led-limit-refresh</code>	60	stabilità, riduce gli artefatti
<code>--led-brightness</code>	50	regolabile da 0 a 100

Serve inoltre la variabile d'ambiente `SPWM_PROFILE_DIR=/home/gillo/rpi-rgb-led-matrix_pwm_experiment/lib/spwm/register-test/data`.

I profili 2, 6, 42 e 68 danno risultato equivalente: sono tutti `Scan_64` della stessa sorgente *P2.5 – FM6373 – DP32020B – 1/64*. Se uno desse problemi, prova gli altri tre.

7.2 Script di prova

Sostituisci il valore di `SLOW` nella seconda riga secondo il modello.

```
cat > ~/dmd.sh << 'EOF'
#!/bin/bash
SLOW=3
export SPWM_PROFILE_DIR=/home/gillo/rpi-rgb-led-matrix_pwm_experiment/lib/spwm/registertest/data
BIN=/rpi-rgb-led-matrix_pwm_experiment/examples-api-use
OPTS="--led-no-drop-privs --led-rows=64 --led-cols=128 --led-chain=2 --led-panel-type=fm6373 --led-
row-addr-type=1 --led-spwm-scan=64 --led-spwm-register-config=2 --led-slowdown-gpio=$SLOW --led-limit-
refresh=60 --led-brightness=50"
sudo -E $BIN/demo -D0 $OPTS
EOF
chmod +x ~/dmd.sh
```

```
~/dmd.sh
```

Atteso: un quadrato intero che ruota e rimbalza, senza duplicazioni né pezzi mancanti. Si esce con `Ctrl+C`.

Se il risultato non è quello, non proseguire: vai all'appendice A, perché installare il DMD Controller sopra un pannello non tarato rende impossibile capire quale dei due livelli non funziona.

Regola d'oro durante i test: power-cycle del pannello tra un tentativo e l'altro, staccando il suo 5V per una decina di secondi (il Pi può restare acceso). I chip S-PWM memorizzano la configurazione nei registri interni, e una configurazione sbagliata può bloccarli fino allo spegnimento.

8. Installazione del DMD Controller da GitHub

Dalla versione 1.5 non serve più trasferire archivi a mano: si installa direttamente dal repository.

```
git clone https://github.com/kWGillo/zedmd-pi.git ~/dmd
```

```
cd ~/dmd && chmod +x *.sh
```

Prima di installare, verifica che i file siano arrivati integri:

```
bash verify.sh
```

Deve rispondere **tutto integro**. Il controllo confronta le impronte md5 di tutti i file: su una scheda in sofferenza un file può arrivare della lunghezza giusta ma con byte nulli al centro, e senza questa verifica il guasto si manifesterebbe solo più tardi, come un errore incomprensibile all'avvio.

```
sudo ./install.sh
```

Lo script esegue in sequenza:

1. installazione dei pacchetti di sistema (Flask, NumPy, Pillow, Cython, font, ffmpeg, Samba)
2. **compilazione dei binding Python** della libreria matrice
3. copia dei file in `/opt/dmd`
4. creazione della configurazione in `/etc/dmd/config.json`
5. creazione della libreria media e della condivisione SMB
6. registrazione del servizio systemd `dmd`

Il passo 2 è lento. Compila da sorgente l'intera libreria C++: da qualche minuto su Pi 4 fino a oltre dieci minuti su Pi Zero 2 W. È normale, non interrompere. Il cursore che gira indica che sta lavorando.

Se la libreria matrice non si trovasse nella home dell'utente:

```
sudo MATRIX_DIR=/percorso/della/libreria ./install.sh
```

9. Configurazione iniziale

9.1 Se hai una configurazione esportata

È la strada breve. Avvia il servizio (§10), apri l'interfaccia web, vai in **Impostazioni → Configurazione**, carica il file JSON esportato dal sistema precedente e premi *Importa e riavvia*. Ritrovi taratura del pannello, colori, fasce orarie e servizi come li avevi lasciati, e puoi saltare il resto di questa sezione.

9.2 Se parti da zero

Apri `/etc/dmd/config.json` e verifica due valori:

Chiave	Valore
<code>panel.slowdown</code>	<code>1</code> Pi Zero W · <code>3</code> Pi Zero 2 W e Pi 3 · <code>5</code> Pi 4
<code>panel.chain</code>	numero di pannelli collegati in cascata

Le altre chiavi rilevanti sono già corrette di default:

Chiave	Valore	Significato
<code>web.port</code>	<code>8080</code>	interfaccia web
<code>web.language</code>	vuoto	lingua dell'interfaccia; vuoto = la decide il browser
<code>zedmd.http_port</code>	<code>80</code>	handshake ZeDMD, non modificare
<code>zedmd.stream_port</code>	<code>3333</code>	frame DMD, TCP e UDP
<code>zedmd.grace_seconds</code>	<code>60</code>	quanto ZeDMD trattiene il display dopo l'ultimo frame
<code>zedmd.client_timeout</code>	<code>10</code>	silenzio oltre il quale il client è considerato caduto
<code>ota.repo</code>	<code>kgGillo/zedmd-pi</code>	repository per l'aggiornamento via rete
<code>air_radar.latitude / longitude</code>	<code>0.0</code>	nessuna posizione preimpostata

Le coordinate del radar restano **soltanto** in questo file: non fanno parte del software e non finiscono in nessun pacchetto o repository.

10. Avvio e interfaccia web

```
sudo systemctl start dmd
```

```
journalctl -u dmd -f
```

Nel log devono comparire le righe del server handshake sulla porta 80, del ricevitore sulla 3333 e dell'interfaccia web sulla 8080. Si esce con `Ctrl+C`: il servizio continua a funzionare.

Il servizio è già abilitato all'avvio automatico. Per verificarlo, riavvia una volta il Raspberry e controlla che riparta da solo.

Interfaccia web: `http://dmdpi.local:8080/` — digitando l'indirizzo senza porta si viene rediretti automaticamente.

10.1 Le tre porte

Porta	Uso	Servita da
80	handshake ZeDMD (cablata nel client, non modificabile)	server HTTP dedicato
3333	frame DMD, TCP e UDP	ricevitore ZeDMD
8080	interfaccia web	Flask

L'handshake **non** passa da Flask di proposito: il client di libzedmd legge la risposta con una sola `recv()` e si

ferma appena riceve meno di 1024 byte. Se header e corpo arrivano in due pacchetti distinti — come fa Flask — il client legge un corpo vuoto, non riconosce il trasporto TCP e ripiega su UDP. Il server dedicato invia tutto in una sola scrittura.

10.2 Le pagine

Impostazioni — luminosità con applicazione immediata, lingua dell'interfaccia, server NTP, fuso orario, ora legale, Night mode e Sleep mode, regolazione fine del driver S-PWM, aggiornamenti, esportazione e importazione della configurazione, indirizzo IP locale, riavvio del servizio.

Orologio — colori indipendenti per ora e data, formato 12 o 24 ore, lingua dei nomi dei giorni (italiano, francese, inglese), lampeggio dei due punti.

Media — libreria, caricamento file, riletture della libreria, anteprima immediata, durate e intervalli, adattamento al pannello, modalità pixel art.

Radar — coordinate e raggio, provider ADS-B, scelta dei parametri di volo, registro CSV dei passaggi con scaricamento, prova diagnostica di una rotta.

Servizi — attivazione dei quattro servizi, sorgente attualmente a schermo e possibilità di forzarne una.

10.3 Lingua

L'interfaccia è in **italiano e inglese**. Alla prima apertura la lingua viene dedotta dal browser; il selettore **IT / EN** in alto a destra la cambia e la scelta viene salvata. Riportando la voce *Lingua dell'interfaccia* su **predefinito** si torna a seguire il browser.

I nomi dei giorni che appaiono **sul pannello** hanno un'impostazione a parte, nella pagina Orologio, e comprendono anche il francese: chi guarda il cabinato non è necessariamente chi configura il sistema.

10.4 Come viene deciso cosa appare sul display

Un solo processo può pilotare i GPIO, quindi tutte le sorgenti vivono nello stesso servizio e un arbitro sceglie chi vince:

Priorità	Sorgente
100	ZeDMD
60	Air Radar
50	Media Player
10	Orologio

ZeDMD prende il controllo **immediatamente** appena un client si connette o arriva un frame, e lo mantiene per `grace_seconds` dopo l'ultimo segnale. Il tempo di grazia evita che l'orologio si intrometta durante le pause normali di Batocera: menu, caricamenti, cambi di schermata.

Sleep mode e Night mode stanno sopra a tutto: Sleep spegne il display, Night ne riduce la luminosità, e Sleep ha la precedenza su Night.

Se Batocera viene spento bruscamente la connessione non si chiude in modo pulito: il ricevitore se ne accorge dopo `client_timeout` secondi di silenzio, poi decorre il tempo di grazia. Con i valori di default il display torna all'orologio entro circa settanta secondi.

11. Collegamento a Batocera

Sulla macchina Batocera, via SSH come `root`:

```
cat > /userdata/system/configs/dmdserver/config.ini << 'EOF'
[DMDServer]
AltColor = 1

[ZeDMD]
Enabled = 0

[ZeDMD-WiFi]
Enabled = 1
WiFiAddr = 192.168.0.XXX
EOF
```

Sostituisci `192.168.0.XXX` con l'indirizzo IP del Raspberry, che trovi nella pagina Impostazioni dell'interfaccia web.

`[ZeDMD] Enabled = 0` disattiva la ricerca del dispositivo su porta seriale: con il collegamento di rete attivo non serve ed eviterebbe conflitti.

Poi, nel menu di Batocera, attiva il servizio **DMD reale** — non “DMD Web”, che è il simulatore su browser.

Verifica

Con `journalctl -u dmd -f` aperto sul Raspberry, riavvia il servizio DMD di Batocera. Deve comparire:

```
[zedmd] client connesso via TCP: ('192.168.0.XXX', ...)
```

Se invece vedi ripetutamente solo `GET /handshake` senza connessione, il client non riesce ad aprire lo stream: verifica che `zedmd.http_port` sia 80 e `web.port` sia 8080.

12. Musica: il brano in ascolto sul pannello

Funzione facoltativa, da fare solo quando il resto funziona. Il Raspberry diventa una cassa AirPlay che non suona: compare fra i dispositivi audio dell'iPhone, accetta il flusso, scarta l'audio e tiene i metadati. Sul pannello compaiono titolo, artista, album e avanzamento del brano. Quello che esce dalle casse vere non cambia.

Siccome al ricevitore AirPlay non importa quale applicazione stia suonando, funzionano allo stesso modo Apple Music, Spotify, Amazon Music e YouTube, senza niente da configurare per ciascuna. Spotify Connect verso casse vere è coperto a parte, tramite l'API di Spotify.

Perché sta qui e non prima

Richiede di compilare `shairport-sync`, che sul Pi 4 porta via un quarto d'ora, e di installare un broker MQTT. Nessuna di queste cose serve ad avere un pannello funzionante: se il DMD non è ancora a posto, torna alla sezione 10 e occupati prima di quello.

Come si fa

Lo script è installato insieme al DMD. Da SSH sul Raspberry:

```
sudo /opt/dmd/setup_nowplaying.sh
```

Chiede quattro cose — il nome con cui comparire fra le casse, e dove sta il broker MQTT (lascia vuoto per installarne uno qui) — e fa tutto il resto: Mosquitto, le dipendenze, `nqptp`, `shairport-sync` compilato con AirPlay 2 e i metadati, la scheda audio fittizia, il file di configurazione, il confinamento ai core 0-2 per non disturbare il pannello, e la sezione MQTT del DMD. In chiusura resta trenta secondi in ascolto: metti musica dal telefono e ti dice se i metadati arrivano davvero.

Lo script è ripetibile. Se una cosa è già fatta lo dice e passa oltre — in particolare, se `shairport-sync` è già compilato a dovere salta del tutto il quarto d'ora di compilazione. Per controllare lo stato senza toccare niente:

```
sudo /opt/dmd/setup_nowplaying.sh --verifica
```

Che cosa resta da fare a mano

- Sul telefono, scegliere il DMD fra le casse: nessuno può farlo al posto tuo.
- Collegare l'account Spotify, se ti serve: richiede un browser e un'applicazione registrata su developer.spotify.com. Si fa dalla pagina **Musica** dell'interfaccia web.
- Le automazioni di Home Assistant, se vuoi coprire anche un HomePod avviato a voce o un Echo — cioè i casi in cui la musica non attraversa il DMD.

Documentazione dedicata

Tutto il resto — che cosa fa lo script passo per passo, la configurazione di Home Assistant, la procedura per

Spotify, e una tabella di diagnosi quando i metadati non arrivano — sta in [docs/now-playing.it.md](#), disponibile anche come PDF. Sono le pagine da aprire se qualcosa non torna; per l'installazione normale bastano i due comandi qui sopra.

13. Aggiornamenti

12.1 Il software DMD, dall'interfaccia web

È la via consigliata. Nella pagina **Impostazioni**, sezione *Aggiornamenti*, il sistema confronta la versione installata con quella pubblicata su GitHub. Quando ce n'è una nuova compare il pulsante di installazione.

L'aggiornamento è costruito per non poter lasciare il sistema rotto:

1. scarica l'archivio del ramo in una cartella temporanea
2. verifica che ci siano tutti i file attesi, che tutto il Python compili e che le impronte md5 corrispondano
3. salva una copia dell'installazione corrente in `/var/lib/dmd/backup`
4. sostituisce i file e riavvia il servizio
5. interroga la web UI per capire se il servizio è davvero ripartito
6. se non risponde, ripristina la copia e riavvia di nuovo

L'esito si legge nel riquadro del registro, in fondo alla stessa pagina.

12.2 Il software DMD, da riga di comando

```
cd ~/dmd && git pull
```

```
bash verify.sh
```

```
sudo ./update.sh
```

`update.sh` verifica il pacchetto prima di toccare l'installazione funzionante e ricontrolla i file copiati prima di riavviare il servizio: se qualcosa non corrisponde si ferma senza fare danni. Adegua anche la configurazione esistente alle chiavi nuove, senza sovrascrivere le tue impostazioni.

L'aggiornamento **non** richiede di ricompilare i binding Python: quel passo si esegue una volta sola in fase di installazione.

Riavvia sempre Batocera dopo un aggiornamento. Il client ZeDMD tiene in memoria lo stato della connessione e la contabilità delle zone già inviate. Dopo il riavvio del servizio sul Raspberry quello stato non è più valido: senza un riavvio di Batocera il display può restare fermo sull'ultimo contenuto o aggiornarsi solo parzialmente.

12.3 Il sistema operativo

```
sudo systemctl stop dmd
```

```
sudo apt update && sudo apt full-upgrade -y
```

```
sudo systemctl start dmd
```

Fermare il servizio non è obbligatorio ma è consigliato: durante l'aggiornamento disco e CPU sono impegnati e il pannello mostrerebbe artefatti.

Se l'aggiornamento tocca il kernel, riavvia al termine.

12.4 La libreria della matrice

Serve di rado — solo se il fork pubblica correzioni utili.

```
sudo systemctl stop dmd
```

```
cd ~/rpi-rgb-led-matrix_pwm_experiment && git pull && make
```

```
sudo pip install . --break-system-packages
```

```
sudo systemctl start dmd
```

L'ultimo passo ricompila i binding Python: senza, il servizio continuerebbe a usare la versione precedente.

12.5 Se l'overlay è attivo

Con la radice in sola lettura (§13.3) nessun aggiornamento sopravvive al riavvio. Vanno disattivato, aggiornato e riattivato:

```
sudo raspi-config nonint disable_overlayfs && sudo reboot
```

```
cd ~/dmd && git pull && sudo ./update.sh
```

```
sudo raspi-config nonint enable_overlayfs && sudo reboot
```

14. Installazione resistente all'usura

Questa sezione nasce da un guasto reale: una scheda SD che ha cominciato ad accettare scritture e a restituire dati diversi. Nessun comando segnalava niente — `scp`, `tar` e `cp` riportavano successo — e i file arrivavano della lunghezza giusta con byte nulli al centro. Il servizio non partiva, e l'unico indizio era un `ValueError` sul primo `import`.

Le quattro contromisure qui sotto costano venti minuti e cambiano l'ordine di grandezza della vita della scheda.

13.1 La scheda giusta

Le sigle grandi sulla confezione — V10, V30, A1, “100 MB/s” — misurano la **velocità**, non la durata. Nessuna dice quanti dati la scheda può scrivere prima di degradarsi, che è il parametro rilevante per un sistema acceso sempre.

Servono schede della linea **high endurance** (SanDisk High Endurance o Max Endurance, Samsung PRO Endurance), nate per dashcam e videosorveglianza, cioè per lo stesso profilo d'uso. Bastano 32 GB: il sistema ne occupa meno di otto e i media stanno altrove. Evita le schede a marchio concesso in licenza, dove il produttore reale del chip non è dichiarato e cambia tra una partita e l'altra.

Le partizioni non proteggono dall'usura. Dedicare una partizione ai media sembra isolarli, ma il *wear leveling* lavora sull'intero chip: le partizioni esistono solo nello spazio degli indirizzi logici, mentre il controller distribuisce le scritture su tutte le celle fisiche. Una partizione separata aiuta il recupero — reinstalli il sistema e i media restano — ma non allunga la vita della scheda di un giorno. A separare davvero è solo un **supporto fisico diverso**.

13.2 Media su chiavetta USB

È l'intervento con il rapporto migliore fra sforzo e risultato: toglie dalla scheda sia le scritture massive sia le letture continue.

```
lsblk
```

Individua la chiavetta (di solito `sda1`) e annota lo UUID:

```
sudo blkid /dev/sda1
```

```
sudo mkdir -p /srv/dmd/media
```

Aggiungi la riga a `/etc/fstab` sostituendo lo UUID e il tipo di filesystem (`exfat` se formattata su Windows o Mac, `ext4` se su Linux):

```
echo 'UUID=xxxx-xxxx /srv/dmd/media exfat defaults,nofail,uid=1000,gid=1000 0 0' | sudo tee -a /etc/fstab
```

```
sudo mount -a && df -h /srv/dmd/media
```

L'opzione `nofail` è importante: senza, il giorno che la chiavetta non c'è il Raspberry si ferma all'avvio invece di proseguire.

13.3 Radice in sola lettura

Raspberry Pi OS può montare la radice in sola lettura, con tutte le scritture dirottate in memoria e scartate al riavvio. Su un sistema che una volta configurato non cambia più è la misura che allunga di più la vita della scheda.

```
sudo raspi-config
```

Performance Options → *Overlay File System* → abilita l'overlay e imposta la partizione di avvio in sola lettura. Poi riavvia.

Il **prezzo da pagare** è che da quel momento nessuna modifica sopravvive al riavvio: né gli aggiornamenti, né le impostazioni cambiate dalla web UI. Per intervenire si disattiva, si lavora, si riattiva — vedi §12.5.

Attivalo **alla fine**, quando pannello, colori e servizi sono a posto. Con l'overlay attivo la web UI accetta le modifiche e le mostra, ma al riavvio tutto torna com'era: un comportamento che fa perdere un pomeriggio a chiunque non se lo ricordi.

13.4 Esportare la configurazione

Dalla pagina Impostazioni, riquadro **Configurazione**. È l'unica parte del sistema che non si può riscaricare: il codice sta su GitHub, la taratura del pannello sta solo qui.

Esportala **dopo ogni modifica importante** e tieni il file su un'altra macchina. La casella *Includi le coordinate del radar* si può togliere quando il file va condiviso o allegato a una segnalazione.

Con quel file, rimettere in piedi tutto su una scheda nuova sono venti minuti; senza, si ricomincia la campagna di prove sul pannello.

15. Righe bianche, rallentamenti e blocchi

Gli stessi sintomi hanno due cause diverse, da distinguere prima di cambiare hardware.

14.1 Righe bianche orizzontali

La libreria genera il segnale del pannello **da programma**, temporizzando i GPIO con precisione di microsecondi. Ogni interruzione del processo — un altro programma, un accesso alla scheda SD, il traffico di rete — allunga il tempo di accensione di una riga, e quella riga appare più chiara. Sono un indicatore di carico, non un guasto del pannello.

Intervento	Dove
<code>isolcpus=3</code> — riserva un core al pannello	<code>/boot/firmware/cmdline.txt</code>
<code>dtparam=audio=off</code> — l'audio usa lo stesso hardware PWM	<code>/boot/firmware/config.txt</code>
<code>panel.slowdown</code> corretto per il modello	pagina Impostazioni
<code>panel.limit_refresh</code> a 60 — oltre costa CPU senza guadagno visibile	pagina Impostazioni
<code>panel.pwm_bits</code> più basso (8 invece di 11) — meno sfumature, molto meno lavoro	pagina Impostazioni
libreria media su chiavetta USB	§13.2

Se compaiono lampi, agisci prima su *cicli extra a fine frame* nella regolazione fine del pannello: aggiunge tempo dopo l'invio dei dati e copre il vuoto che genera il lampo. Parti da 1 e sali di uno alla volta.

14.2 Blocchi, `Bus error`, `Input/output error`

Messaggi come questi **non** sono un problema di velocità:

```
Bus error
-bash: /usr/bin/nice: Input/output error
cat: command not found
```

Il sistema non riesce più a **leggere i propri programmi dal disco**. Un Raspberry lento è lento, non perde l'accesso a `/usr/bin`.

```
vcgencmd get_throttled
```

```
dmesg | grep -i -E "under-voltage|mmc|ext4|i/o error" | tail -20
```

Diverso da `0x0` sul primo → **alimentazione**: bit 0 acceso significa sottotensione in corso, bit 16 che si è verificata almeno una volta dall'accensione. Rimedio: alimentare il Pi separatamente dai pannelli, con la massa in comune.

Righe `mmcblk0: error -110` o `EXT4-fs error` sul secondo → **scheda SD in esaurimento**. Rimedio: scheda nuova high endurance e reinstallazione, seguendo la §13 per non ritrovarsi nella stessa situazione.

Un altro segnale inequivocabile: file che arrivano corrotti in punti **diversi** a ogni trasferimento. Un pacchetto sbagliato rompe sempre lo stesso file; un supporto che cede ne rompe uno diverso ogni volta. `verify.sh` lo rende visibile subito.

Passare a un Raspberry Pi 4 **non risolve** il problema della scheda: la SD resta la stessa. Risolve il primo, perché ha più CPU e può avviarsi da SSD USB 3.0, togliendo del tutto la scheda dal percorso.

14.3 Librerie media molto grandi

Le raccolte Pixelcade complete arrivano a decine di migliaia di file.

Dalla versione 1.6 l'elenco viene tenuto in memoria per cinque minuti invece di essere riletto a ogni contenuto e a ogni richiesta della web UI: con 40.000 file quella scansione continua era essa stessa una causa di righe bianche. Dopo aver copiato file dalla condivisione di rete, usa il pulsante **Rileggi la libreria** nella pagina Media.

Per le copie massive, ferma il servizio e abbassa la priorità:

```
sudo systemctl stop dmd
```

```
nice -n 19 ionice -c3 tar xzf pixelcade.tar.gz -C /srv/dmd/media && sync
```

```
sudo systemctl start dmd
```

16. Risoluzione problemi

15.1 Pannello

Sintomo	Causa	Rimedio
unable to open register-profile catalog	la libreria abbandona i privilegi di root e daemon non legge <code>/home/gillo</code>	aggiungere <code>--led-no-drop-privs</code>
unable to locate fm6373.profiles	variabile d'ambiente assente	impostare <code>SPWM_PROFILE_DIR</code> (con <code>sudo -E</code> se esportata)
Immagine duplicata verticalmente	<code>--led-spwm-scan=32</code>	usare <code>--led-spwm-scan=64</code>
Barre colorate scombinare, immagine spezzata	profilo di registro sbagliato	<code>--led-spwm-register-config=2</code> (o 6/42/68)
Pannello nero dopo test falliti	i chip S-PWM restano in stato inconsistente	power-cycle del pannello, 10 secondi senza 5V
Solo metà pannello acceso	<code>--led-spwm-data-layout 4 o 5</code>	non usarlo: il default è 0
Sfarfallio, righe sporadiche	slowdown errato per il modello	verificare <code>\$SLOW</code> secondo §1.2

Immagine instabile su Pi 4	slowdown troppo basso	il Pi 4 richiede 5 , non 3
Comportamenti erratici su Pi 4	alimentatore insufficiente	USB-C 5V/3A di qualità

15.2 Servizio e interfaccia

Sintomo	Causa probabile	Rimedio
Il servizio non parte, errore su <code>rgbmatrix</code>	binding Python non compilati	rilanciare <code>install.sh</code>
Pannello nero ma servizio attivo	nessuna sorgente abilitata	attivare Orologio dalla pagina Servizi
<code>Address already in use</code> sulla porta 80	un altro web server è attivo	fermarlo (<code>lighttpd</code> , <code>apache2</code> , <code>nginx</code>)
Handshake ripetuto, nessun frame	porte scambiate	<code>zedmd.http_port = 80</code> , <code>web.port = 8080</code>
Web UI assente, pannello acceso	errore all'import di <code>webui</code>	<code>journalctl -u dmd -n 30</code> , poi <code>bash ~/dmd/verify.sh /opt/dmd</code>
<code>ValueError: source code string cannot contain null bytes</code>	file corrotto in scrittura	\$14.2
L'orologio interrompe Batocera	tempo di grazia troppo corto	alzare <code>zedmd.grace_seconds</code>
Dopo un aggiornamento il display resta fermo	stato del client non più valido	riavviare Batocera
I file copiati via SMB non compaiono	elenco ancora in cache	pulsante <i>Rileggi la libreria</i>
Le modifiche spariscono al riavvio	overlay attivo	\$12.5
Il radar non mostra nulla	coordinate a 0	inserirle nella pagina Radar
La rotta non appare	volo assente dal servizio rotte	prova diagnostica nella pagina Radar

17. Comandi utili e struttura dei file

```

sudo systemctl start dmd      # avvia
sudo systemctl stop dmd      # ferma
sudo systemctl restart dmd   # riavvia
systemctl status dmd         # stato
journalctl -u dmd -f         # log in tempo reale
journalctl -u dmd -n 100     # ultime 100 righe
curl http://localhost/handshake # verifica dell'handshake
vcgencmd get_throttled       # 0x0 = alimentazione a posto
bash ~/dmd/verify.sh /opt/dmd # integrità dei file installati
cd ~/dmd && git pull          # scarica l'ultima versione

```

<code>/opt/dmd/dmdd.py</code>	servizio principale, arbitro e ciclo di rendering
<code>/opt/dmd/display.py</code>	proprietario esclusivo del pannello
<code>/opt/dmd/dmdconf.py</code>	configurazione persistente
<code>/opt/dmd/webui.py</code>	interfaccia web (Flask)
<code>/opt/dmd/i18n.py</code>	testi in italiano e inglese
<code>/opt/dmd/ota.py</code>	aggiornamento via rete da GitHub
<code>/opt/dmd/zedmd_http.py</code>	server dell'handshake ZeDMD sulla porta 80
<code>/opt/dmd/sources/</code>	sorgenti di contenuto
<code>/etc/dmd/config.json</code>	configurazione
<code>/var/lib/dmd/backup/</code>	copia dell'installazione precedente
<code>/var/lib/dmd/config-*.json</code>	copie della configurazione prima di un'importazione
<code>/var/lib/dmd/flights.csv</code>	registro dei passaggi aerei
<code>/var/lib/dmd/ota.log</code>	registro degli aggiornamenti
<code>/srv/dmd/media/</code>	libreria media, condivisa via SMB
<code>/etc/systemd/system/dmd.service</code>	

18. Appendice A — diagnostica del pannello

Serve solo se il quadrato rotante di §7.2 non funziona, o se in futuro cambiano i pannelli.

A.1 Ricerca del profilo di registro

Demo interattiva che scorre i 77 profili disponibili per FM6373:

```
export SLOW=3
sudo SPWM_PROFILE_DIR=/home/gillo/rpi-rgb-led-matrix_pwm_experiment/lib/spwm/register-test/data \
~/rpi-rgb-led-matrix_pwm_experiment/examples-api-use/demo -D15 --led-no-drop-privs \
--led-rows=64 --led-cols=128 --led-chain=1 --led-panel-type=fm6373 \
--led-spwm-row-addr-type=1 --led-spwm-scan=64 --led-slowdown-gpio=$SLOW \
--led-limit-refresh=60 --led-brightness=50
```

Frecce **sinistra/destra** per scorrere, **M** per marcare un profilo come buono. Il numero mostrato si riusa in `--led-spwm-register-config=N`.

A.2 Programma `canvas-map`

Colora il canvas a bande, per capire come i pixel logici finiscono su quelli fisici. Crea `~/canvas-map.cc` con `nano` — incollare blocchi lunghi via heredoc può corrompersi:

```
#include "led-matrix.h"
#include <signal.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

using namespace rgb_matrix;

volatile bool run_flag = true;
static void stop(int) { run_flag = false; }

int main(int argc, char **argv) {
    RGBMatrix::Options opt;
    rgb_matrix::RuntimeOptions rt;
    if (!ParseOptionsFromFlags(&argc, &argv, &opt, &rt)) return 1;
    const char *mode = (argc > 1) ? argv[1] : "cols";
    RGBMatrix *m = RGBMatrix::CreateFromOptions(opt, rt);
    if (m == NULL) return 1;
    FrameCanvas *c = m->CreateFrameCanvas();
    int W = c->width(), H = c->height();
    printf("Canvas: %dx%d, mode=%s\n", W, H, mode);
    if (strcmp(mode, "axes") == 0) {
        for (int x = 0; x < W; x++) c->SetPixel(x, 0, 200, 0, 0);
        for (int y = 0; y < H; y++) { c->SetPixel(0, y, 0, 200, 0); c->SetPixel(1, y, 0, 200, 0); }
        c->SetPixel(0, 0, 255, 255, 255);
    } else if (strcmp(mode, "pair") == 0 && argc > 3) {
        int px = atoi(argv[2]);
        int py = atoi(argv[3]);
        c->SetPixel(px, py, 255, 0, 0);
        c->SetPixel(px + 1, py, 0, 255, 0);
        c->SetPixel(px + 2, py, 0, 0, 255);
    } else {
        for (int x = 0; x < W; x++) {
            for (int y = 0; y < H; y++) {
                uint8_t r = 0, g = 0, b = 0;
                int band;
                if (strcmp(mode, "fine") == 0) {
                    band = (x / 8) % 3;
                    if (band == 0) r = 180;
                    else if (band == 1) g = 180;
                    else b = 180;
                } else {
                    if (strcmp(mode, "cols") == 0) band = x * 4 / W;
                    else band = y * 4 / H;
                    if (band == 0) r = 180;
                    else if (band == 1) g = 180;
                    else if (band == 2) b = 180;
                    else { r = 180; g = 180; }
                }
                c->SetPixel(x, y, r, g, b);
            }
        }
        c = m->SwapOnVSync(c);
        signal(SIGINT, stop);
        while (run_flag) usleep(100000);
        m->Clear();
        delete m;
        return 0;
    }
}
```

```
g++ -O2 -o ~/canvas-map ~/canvas-map.cc \
-I ~/rpi-rgb-led-matrix_pwm_experiment/include \
```

```
-L ~/rpi-rgb-led-matrix_pwm_experiment/lib \
-lrgbmatrix -lrt -lm -lpthread
```

Modalità disponibili come primo argomento:

- `cols` — quattro bande verticali (rosso/verde/blu/giallo), mappa le colonne
- `rows` — quattro bande orizzontali, mappa le righe
- `fine` — strisce verticali da 8 px, misura il fattore di scala
- `axes` — riga 0 in rosso, colonne 0-1 in verde: rivela l'orientamento
- `pair X Y` — tre pixel adiacenti R/G/B: rivela duplicazioni e sfasamenti

```
export SLOW=3
sudo SPWM_PROFILE_DIR=/home/gillo/rpi-rgb-led-matrix_pwm_experiment/lib/spwm/register-test/data \
~/canvas-map rows --led-no-drop-privs --led-rows=64 --led-cols=128 --led-chain=1 \
--led-panel-type=fm6373 --led-spwm-row-addr-type=1 --led-spwm-scan=64 \
--led-spwm-register-config=2 --led-slowdown-gpio=$SLOW --led-limit-refresh=60 --led-brightness=50
```

Configurazione corretta = quattro bande di quattro colori diversi, ciascuna una sola volta.

19. Appendice B — pubblicare una nuova versione su GitHub

Da eseguire **sul Mac**, non sul Raspberry. Incolla **un comando alla volta**: quando un `cd` fallisce in una sequenza incollata tutta insieme, i comandi successivi vengono eseguiti nella cartella sbagliata.

La prima volta serve il client GitHub:

```
brew install gh && gh auth login
```

Scompatta il pacchetto ricevuto:

```
cd ~/Downloads
```

```
ls ~/Downloads/*.tar.gz
```

In `zsh` un carattere jolly che non trova nulla fa fallire il comando con `no matches found`: leggi il nome vero invece di indovinarlo.

```
tar xzf zedmd-pi.tar.gz
```

La cartella scompattata **non contiene la cronologia git**, quindi non fare `git init` al suo interno: creerebbe una storia senza parentela con quella già pubblicata e il push verrebbe rifiutato. Si parte dal repository vero:

```
git clone https://github.com/kwGillo/zedmd-pi.git zedmd-pi-repo
```

```
cd ~/Downloads/zedmd-pi-repo
```

```
find . -mindepth 1 -maxdepth 1 -not -name .git -exec rm -rf {} +
```

```
cp -R ~/Downloads/zedmd-pi/. .
```

Il punto dopo la barra è indispensabile: significa “il contenuto della cartella”, non la cartella stessa.

```
pwd && grep __version__ version.py && git remote -v
```

```
git add -A
```

```
git commit -m "<versione>: <cosa è cambiato>"
```

```
git push
```

```
curl -s https://raw.githubusercontent.com/kwGillo/zedmd-pi/main/version.py | grep __version__
```

Quest'ultimo è l'indirizzo che interroga l'aggiornamento automatico del Raspberry: se qui vedi il numero nuovo, l'OTA lo vedrà.

Le volte successive `zedmd-pi-repo` resta sul Mac con il suo `.git`: bastano `git pull`, svuota, ricopia, commit, push.

20. Riferimenti

- Progetto: <https://github.com/kwGillo/zedmd-pi>
- Fork con supporto S-PWM: https://github.com/kingdo9/rpi-rgb-led-matrix_pwm_experiment
- Guida al tuning S-PWM: `spwm.md` nel repository del fork
- Libreria originale: <https://github.com/hzeller/rpi-rgb-led-matrix>
- Discussione sui chip S-PWM: <https://github.com/hzeller/rpi-rgb-led-matrix/issues/1866>
- Protocollo ZeDMD: <https://github.com/PPUC/libzedmd>
- Lato Batocera: <https://github.com/vpinball/libdmdutil>

Specifiche dell'hardware

Pannelli: P2.5 indoor, 128×64 px, 320×160 mm, driver **FM6373** (S-PWM), row driver DP32020B, linee di indirizzo A/B/C (D ed E = NC), interfaccia HUB75.

Risoluzione finale: 256×64 px con i due pannelli in cascata.

Controller: qualsiasi Raspberry Pi fra quelli elencati in §1.2. Altre schede a scheda singola non sono utilizzabili: la libreria accede direttamente ai registri hardware Broadcom/RP1.